DATA 211 – Introduction to Data Science

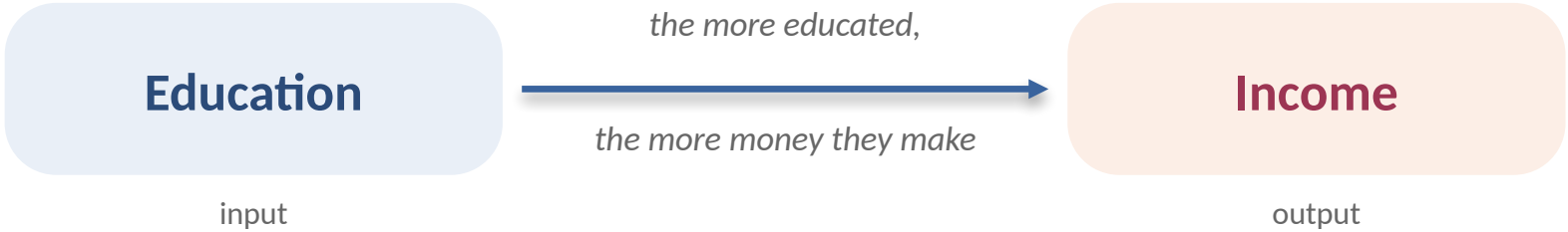
LINEAR REGRESSION

Prof Dr Usman Qamar

Regression

A process for modeling the **relationship** between variables of interest.

Example



- If we know the relationship between education and income, we can **predict** someone's income from their education.
- Simply speaking, learning such a relationship *is* regression.

Correlation or Regression?

Question A

Data for 10 students who took an exam last semester: the student ID, the student's **attitude** before the exam, and the **score** obtained.

1. Calculate the correlation between Attitude and Score using Python.
2. If attitude decreases, then score will increase. True or False? Explain using Part 1.
3. If a new student's attitude is 74 units, can you estimate his score before the exam using correlation?

STU-ID	Attitude	Score
1	65	129
2	67	126
3	68	143
4	70	156
5	71	161
6	72	158
7	72	168
8	73	166
9	73	182
10	75	201

Regression-1.csv

Question A: Answers

```python
# 1. Calculate the correlation between Attitude and Score
import pandas as pd
df = pd.read_csv('data/Regression-1.csv', delimiter =',')
corr = df.drop(['STU-ID'], axis=1).corr()
display(corr)
```

	Attitude	Score
Attitude	1.00000	**0.94179**
Score	**0.94179**	1.00000

r = 0.94

very strong, positive

1 **Correlation** — r = 0.9418: a very strong positive correlation.

2 **False** — The direction of the relationship is positive: if attitude decreases, the score decreases.

3 **No** — Correlation shows the strength and direction of a relationship, but it cannot depict the actual relation, so it cannot estimate a score.

Regression vs Correlation

Correlation

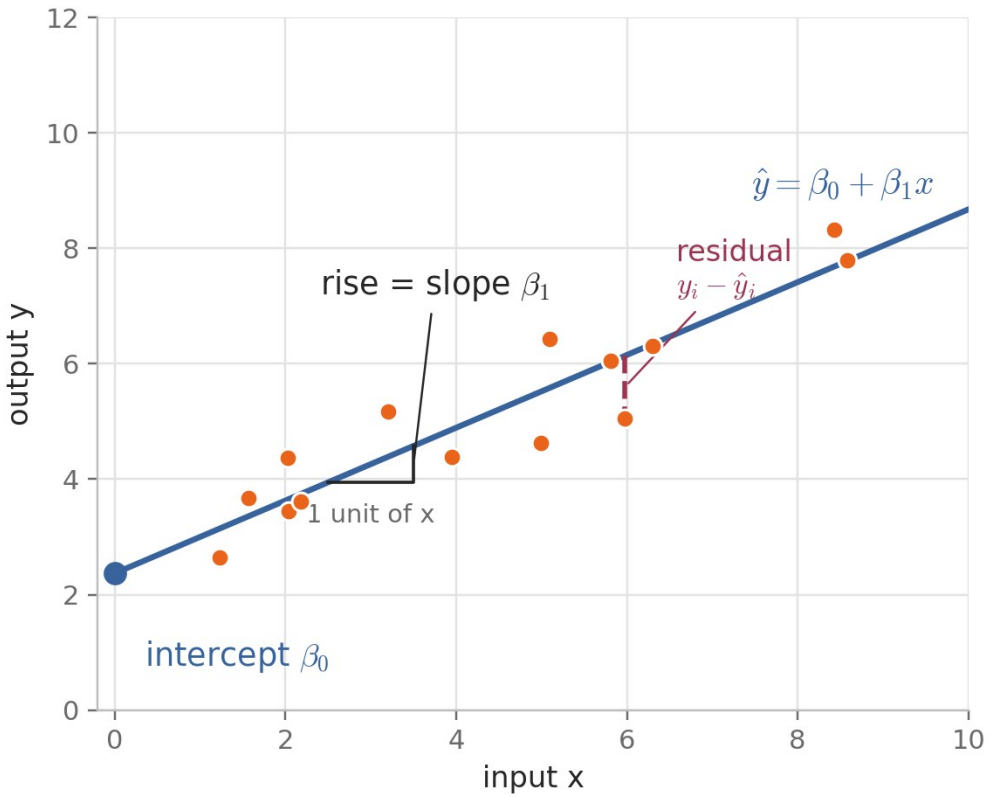
- May indicate **whether** two variables are related or not.
- Does not provide information on **how** one variable is related to another.

Regression

- May identify how one or more variables are related to an output variable.
- Gives details of how the input variables affect the output variable.
- Beyond estimating a relationship, it is a way of **predicting** an output from one or more inputs.

Correlation: *are they related?* Regression: *how, and what will y be?*

Linear Regression Overview



Illustrative data.

- Assumes the relationship between the inputs and the output is **linear**: it can be expressed as a line or a hyperplane.
- **Intercept β_0**: the value of ŷ when x = 0.
- **Slope β_1**: the change in ŷ for each one-unit increase in x.
- **Residual**: the gap between an observed y and the line's prediction ŷ.

Fitting the line means choosing β_0 and β_1 so that the residuals are as small as possible.

Single Input Single Output (SISO)

How is one input variable related to one output variable?

Notation

x is the input variable and *y* is the output variable.

Model

$$y = \beta_0 + \beta_1 x$$

- β_1 is the slope of x, and β_0 is the intercept of the equation.

Goal

- Linear regression estimates the best values of β_0 and β_1.
- When a new data point x arrives with an unknown y, the estimated β_0 and β_1 give an estimated value $\hat{y}$.
- **The goal is to have $\hat{y}$ as close as possible to y.**

Solution Method for SISO Linear Regression

From statistical analysis, the coefficients can be estimated as follows:

Slope

$$\beta_1 = r \frac{sd_y}{sd_x}$$

change in y for each 1-unit increase in x

Intercept

$$\beta_0 = \overline{y} - \beta_1 \overline{x}$$

value of y when x = 0

where

Symbol	Meaning
r	Pearson's correlation coefficient
sd_x, sd_y	standard deviations of x and y
$\bar{x}$, $\bar{y}$	means of x and y

Example: SISO Linear Regression

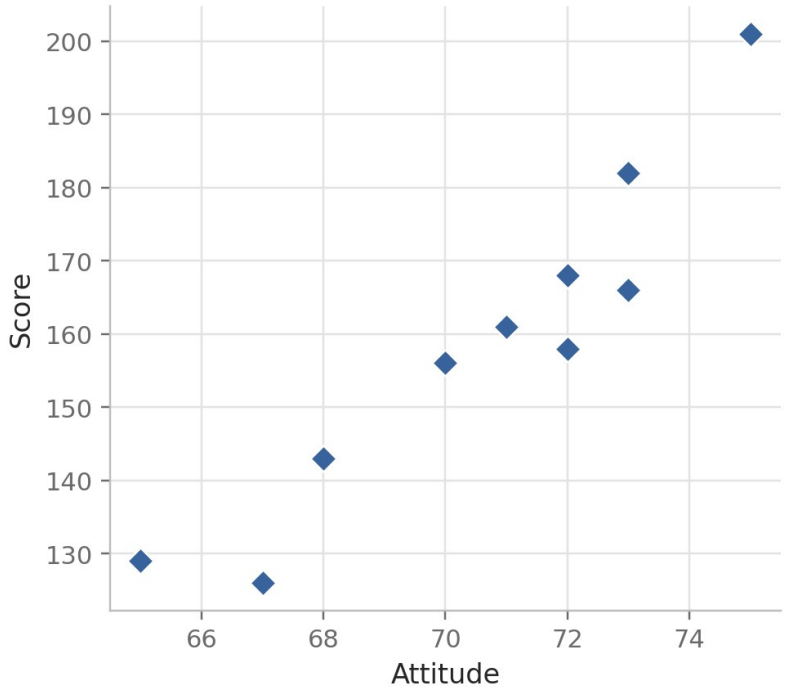
Question B Consider the data from Question A (Regression-1.csv).

1. Visualize the relationship between Attitude and Score (Attitude on the x-axis, Score on the y-axis).
2. Is the relationship linear? Comment.
3. Identify the linear relationship between Attitude and Score using the formula. Then verify the result using Python.
4. If a new participant with a positive attitude of 78 takes the exam, what is the estimated score?
5. If new participants with attitudes of 78, 74, 68 and 69 take the exam, what are their estimated scores?
6. The estimated and actual scores of the student with attitude 68 differ. Is there an error in the approach?

QUESTION B.1–2: VISUALIZE THE RELATIONSHIP

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data/Regression-1.csv',
                 delimiter =',')
plt.figure()
sns.relplot(x='Attitude', y='Score',
            color = 'purple', marker = 'D',
            kind='scatter', data=df)
plt.show()
```



B.2 Is the relationship linear?

From the plot, the relationship could be linear: the points rise together roughly along a straight line.

Question B.3: Estimate the Coefficients

By the formula

r	mean Attitude	mean Score	SD Attitude	SD Score
0.9418	**70.6**	**159**	**2.94**	**21.64**

$$\beta_1 = 0.9418 \times \frac{21.64}{2.94} = 6.93 \qquad \beta_0 = 159 - (6.93 \times 70.6) = -330.26$$

Verify in Python

Python

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression(fit_intercept=True)
reg.fit(df[['Attitude']], df['Score'])      # single column
print(f'Beta1, the slope is {np.round(reg.coef_,2)}; '
      f'and Beta0, the intercept is {np.round(reg.intercept_,2)}')
```

Output

```
Beta1, the slope is [6.93]; and Beta0, the intercept is -330.46
```

−330.26 vs −330.46: rounding β_1 to 6.93 before computing β_0.

Question B.4–5: Predict New Scores

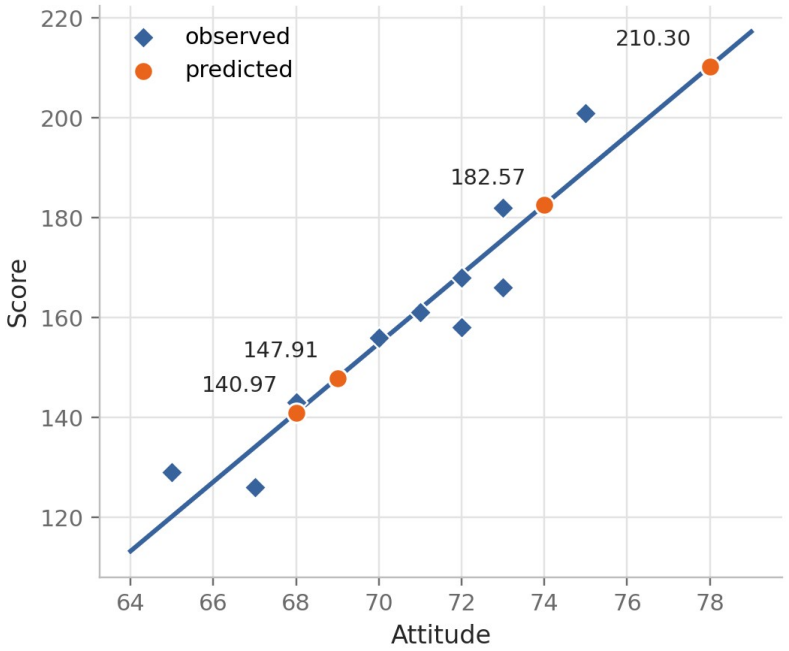
B.4 by hand, attitude 78

$$\hat{y} = -330.46 + (6.93 \times 78) = 210.08$$

Python

```
x_new = 78
y_new = reg.predict(np.array(x_new).reshape(1, -1))

x_new = [78,74,68,69]
y_new = reg.predict(np.array(x_new).reshape(-1, 1))
print(np.round(y_new,2).tolist())
```



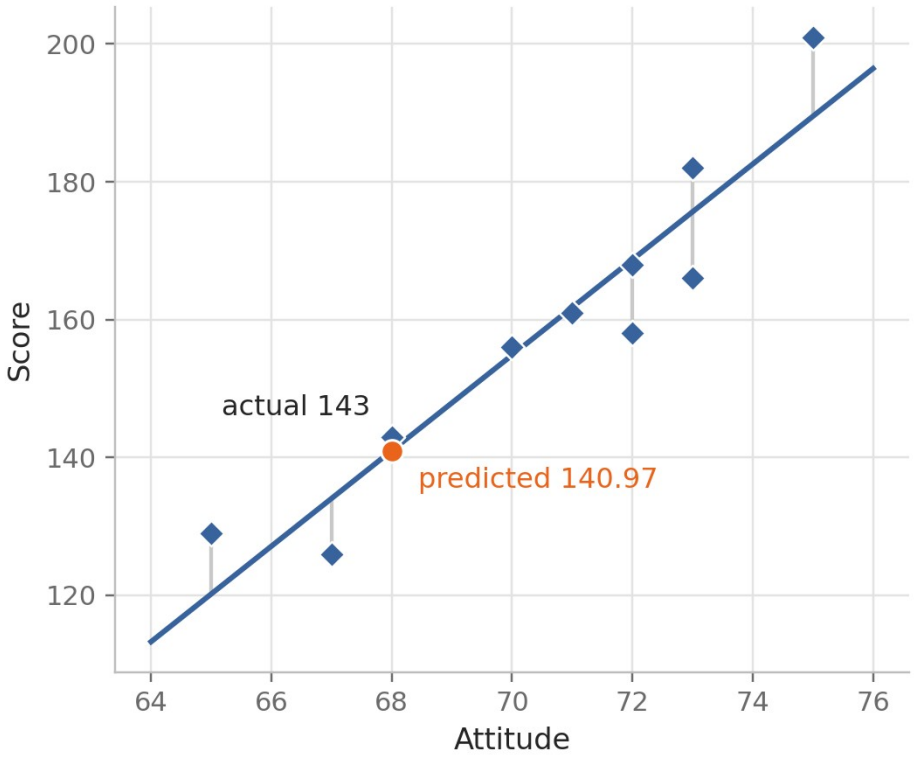
Attitude	78	74	68	69
Estimated score	210.30	182.57	140.97	147.91

By hand 210.08 vs Python 210.30: again the rounding of β_1.

Tip: the UserWarning

'X does not have valid feature names' appears because the model was fitted on a DataFrame. Predict on pd.DataFrame({'Attitude': x_new}) to avoid it.

Question B.6: Why Do Estimates Differ?



143

actual score

140.97

estimated score

No, the approach is not in error.

- The relationship is not perfectly linear, so a straight line cannot pass through every point. Some error remains.
- Linear regression chooses the line that **minimizes the mean squared error** over the data points.

Grey lines are the residuals. Their mean square for this fit is **52.92**: no other straight line does better on these ten students.

EXAMPLE 2: SISO LINEAR REGRESSION

Question C Consider the data given in `Regression-2.csv`.

1. Read and display the data.
2. Plot the data.
3. Can we model the data using a linear relationship? If yes, suggest the linear model and highlight the unknowns.

```python
df = pd.read_csv('data/Regression-2.csv', delimiter =',')
display(df.T)
df.describe().T
```

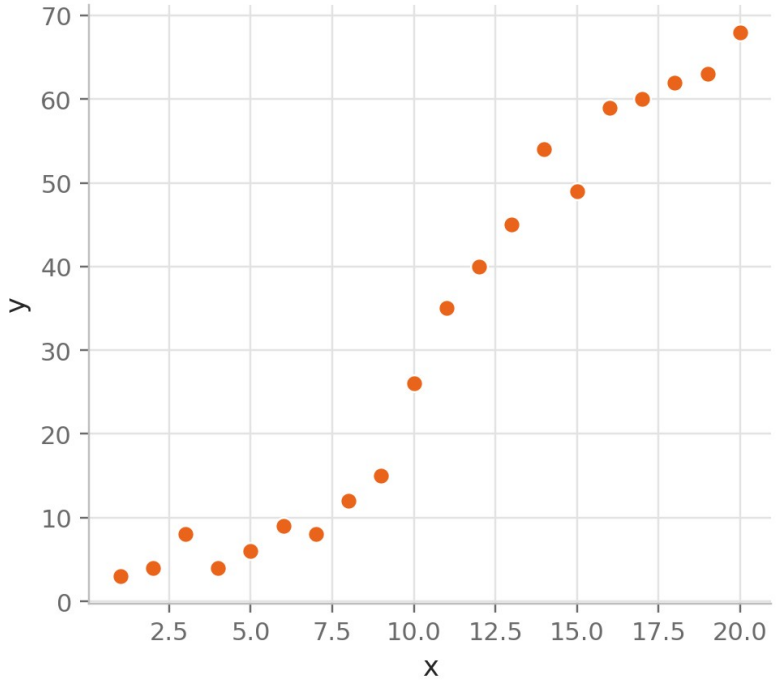
x	1	2	3	4	5	6	7	8	9	10
y	3	4	8	4	6	9	8	12	15	26

x	11	12	13	14	15	16	17	18	19	20
y	35	40	45	54	49	59	60	62	63	68

20	10.5 / 5.92	31.5 / 24.25	3 – 68
observations	x: mean / std	y: mean / std	y: min – max

Question C.2–3: Plot and Model the Data

```python
plt.figure()
sns.relplot(x='x', y='y', color = 'r',
            marker = 'o', kind='scatter', data=df)
plt.show()
```



C.3 The data can be modeled with a linear relationship (more or less).

Denote the data as (x_i, y_i), i = 1, …, n, where n is the number of observations.

$$y = \beta_1 x + \beta_0$$

Two unknowns

- β_1: the line's slope
- β_0: the line's y-intercept

How to Find the Coefficients?

Idea

1. Try different values of β_1 and β_0.
2. Find the best values.

Mechanism

1. Define an *error function* (a.k.a. cost function) that measures how **good** a given line is.
2. For given β_1, β_0 it returns an error value.

Squared error function

$$\epsilon = \frac{1}{n}\sum_{i=1}^{n}((\beta_1 x_i + \beta_0) - y_i)^2$$

- A classical way to measure the error: the sum of the squared error of each point.
- Linear regression with this error function is **Ordinary Least Squares (OLS)**.

Solution method

The values of β_1 and β_0 that **minimize** the error function give the best line. Minimization can use different algorithms; we focus on **gradient descent**.

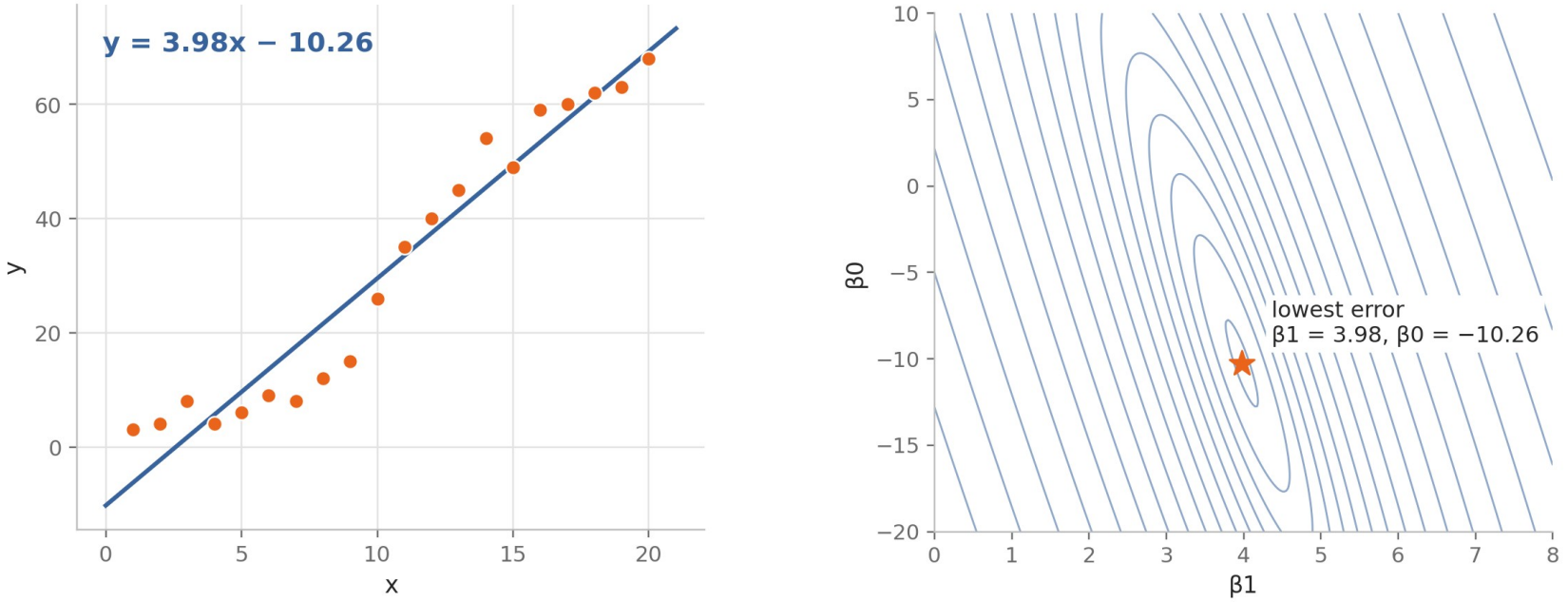
What Are the Best Coefficient Values?

Python

```
reg = LinearRegression(fit_intercept=True)
reg.fit(df[['x']], df['y'])      # single column
best_b1 = reg.coef_[0]
best_b0 = reg.intercept_
print(f'The best values for b1 and b0 are {np.round(best_b1,2)} and '
      f'{np.round(best_b0,2)} respectively.')
```

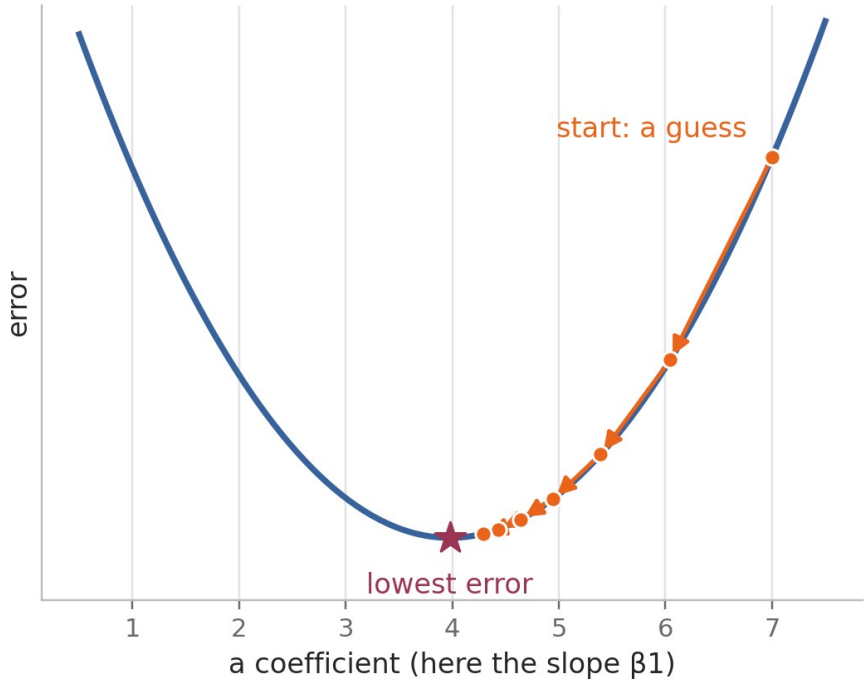
Output

```
The best values for b1 and b0 are 3.98 and -10.26 respectively.
```



Right: the best values fall at the lowest point of the error function.

Gradient Descent: The Idea



Error for different slopes, with the intercept at its best value.

1 Start

Pick a guess for β_1 and β_0 and calculate the error.

2 Find downhill

The gradient (slope of the error) shows which way the error falls.

3 Step

Move β_1 and β_0 a small step downhill. The step size is λ.

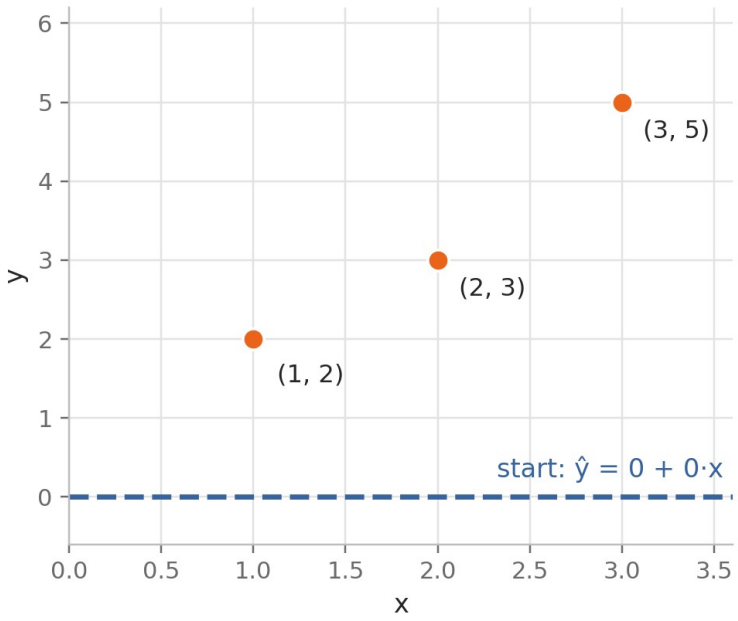
4 Stop

Repeat until the error no longer falls.

Why it works here

The squared error is bowl-shaped (convex), so walking downhill always ends at the single lowest point. Keep λ small: a large step can overshoot the bottom.

Worked Example: Gradient Descent by Hand



x	y
1	2
2	3
3	5

Three points, small enough to follow by hand.

Goal: the best line $\hat{y} = \beta_0 + \beta_1 x$.

$\beta_0 = 0$
start intercept

$\beta_1 = 0$
start slope

$\lambda = 0.1$
step size

The starting line $\hat{y} = 0$ is flat on the x-axis: far below every point.

Each iteration does four things

1 Predict

$\hat{y} = \beta_0 + \beta_1 x$ for every point

2 Measure

error = $\hat{y} - y$; MSE = average of error^2

3 Gradients

slope: 2 × average of (error × x)

intercept: 2 × average of error

4 Update

new β = old β − λ × gradient

Iteration 1, Step by Step

Current line: $\hat{y} = 0 + 0 \cdot x$

x	y	$\hat{y}$	error $\hat{y} - y$	error²	error × x
1	2	0	−2	4	−2
2	3	0	−3	9	−6
3	5	0	−5	25	−15
average			**−3.33**	**12.67**	**−7.67**

The average of error² is the MSE: 12.67.

Gradients

slope: $2 \times (-7.67) = \mathbf{-15.33}$

intercept: $2 \times (-3.33) = \mathbf{-6.67}$

Update ($\lambda = 0.1$)

$\beta_1 = 0 - 0.1 \times (-15.33) = \mathbf{1.53}$

$\beta_0 = 0 - 0.1 \times (-6.67) = \mathbf{0.67}$

New line: $\hat{y} = 0.67 + 1.53x$

MSE 12.67 → 0.22

Iteration 2: A Smaller Step Back

Current line: $\hat{y} = 0.67 + 1.53x$

x	y	$\hat{y}$	error $\hat{y} - y$	error^2	error × x
1	2	2.20	0.20	0.04	0.20
2	3	3.73	0.73	0.54	1.47
3	5	5.27	0.27	0.07	0.80
average			**0.40**	**0.22**	**0.82**

Gradients

slope: $2 \times 0.82 = \mathbf{1.64}$

intercept: $2 \times 0.40 = \mathbf{0.80}$

Update ($\lambda = 0.1$)

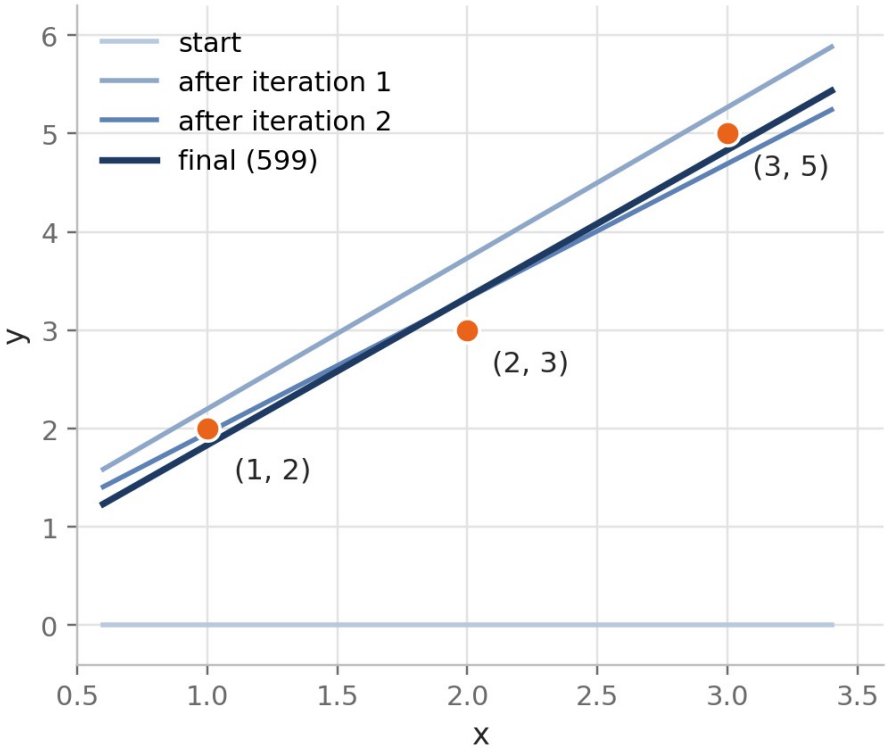
$\beta_1 = 1.53 - 0.1 \times 1.64 = \mathbf{1.37}$

$\beta_0 = 0.67 - 0.1 \times 0.80 = \mathbf{0.59}$

- Now every error is **positive**: the first step went slightly too far. Positive gradients pull both coefficients back down, by a much smaller amount.
- MSE 0.22 → 0.07. Steps shrink as the error bowl flattens near the bottom.

Watching It Converge

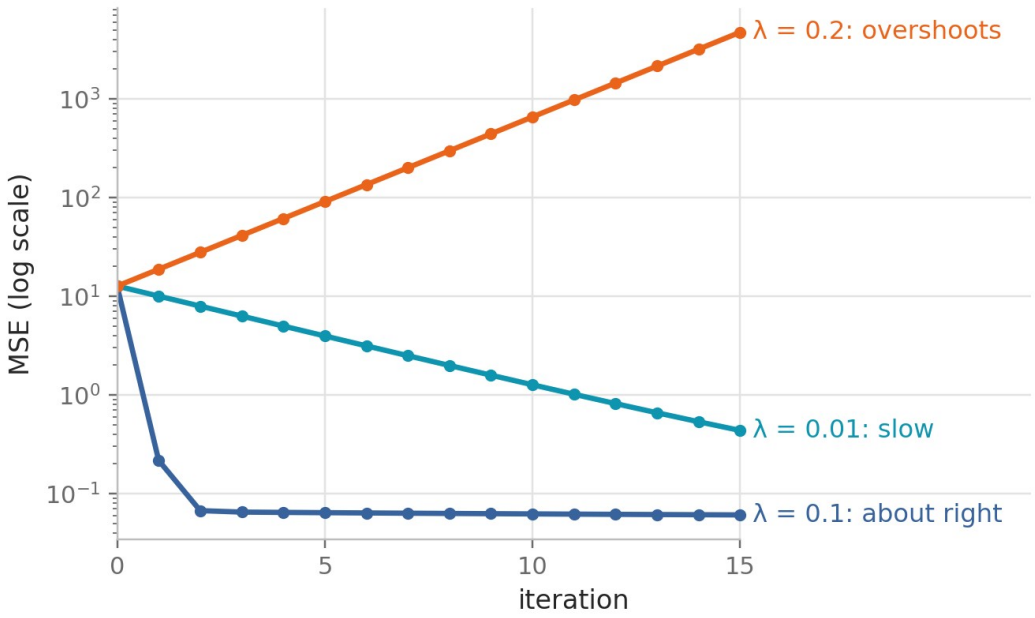
Iteration	β_0	β_1	MSE
0 (start)	0.000	0.000	12.667
1	0.667	1.533	0.216
2	0.587	1.369	0.067
3	0.588	1.390	0.065
5	0.576	1.393	0.064
10	0.548	1.406	0.062
50	0.414	1.464	0.056
100	0.357	1.489	0.056
200	0.335	1.499	0.056
599 (stop)	**0.333**	**1.500**	**0.056**



The notebook's stopping rule ends after 599 iterations at $\hat{y} = 0.333 + 1.500x$.

LinearRegression gives exactly the same line: gradient descent found the least-squares answer.

Choosing the Step Size



λ = 0.01: too small

The error falls steadily but slowly: many more iterations.

λ = 0.1: about right

Big early progress, then it settles at the minimum.

λ = 0.2: too large

Each step overshoots the bottom by more than the last: the error grows.

Same data, same start, same 15 iterations: only λ changes.

The notebook's rule (stop when the error stops falling) protects against a too-large λ: it stops at once, but on a bad line. Start small and increase λ if progress is slow.

MISO Linear Regression

So far: one input. In practice we usually have **several** input variables, say P of them, and one output.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_P x_P$$

- β_0 is the intercept; $\beta_1 \ldots \beta_P$ give one coefficient per input.
- Each coefficient is the change in $\hat{y}$ when its input rises by one unit, with the other inputs held fixed.
- The error is the same mean squared error as before, averaged over all n observations.

Two ways to find the coefficients

Gradient descent

The same algorithm as for SISO: at each step, move every coefficient a small step downhill.

A direct formula

Least squares also has a closed-form solution that gives all coefficients at once.

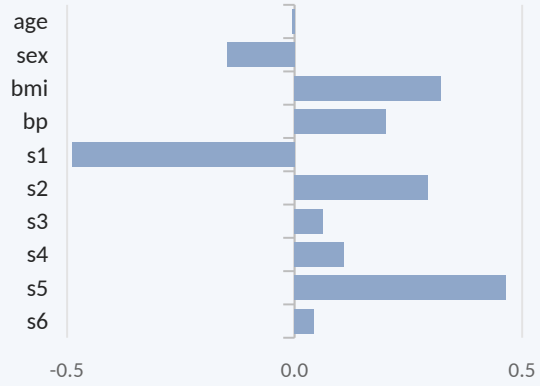
Ridge and Lasso: What Are They?

Two versions of linear regression that add a **penalty for large coefficients**. They pull the coefficients towards zero to stop the model over-fitting.

OLS

ordinary least squares

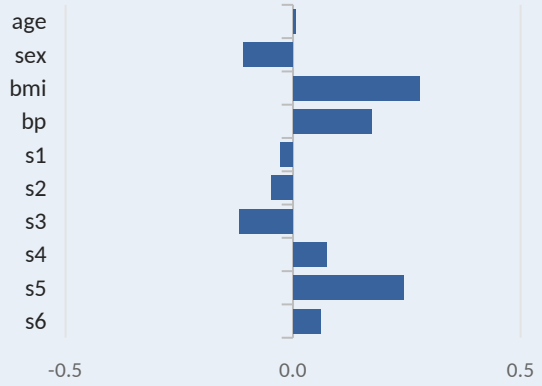
Fits the training data as closely as it can. With many or correlated inputs, coefficients can grow large and unstable.



Ridge

L2 penalty

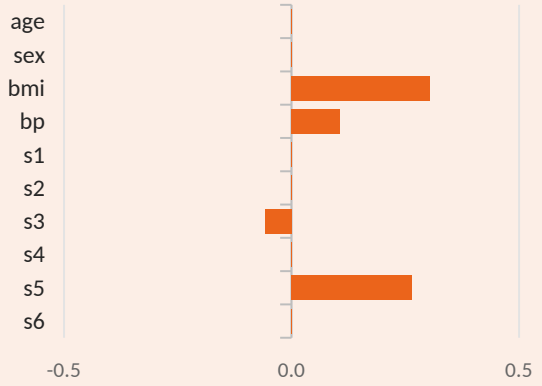
Shrinks every coefficient towards zero but keeps all inputs. Most useful when inputs are correlated.



Lasso

L1 penalty

Shrinks, and sets some coefficients exactly to zero: those inputs are dropped (feature selection).



Penalized Linear Regression (PLR)

- Coefficients that minimize the squared error on the training data may not perform well on new data. This is **over-fitting**.
- To control it, add a **penalty** for large coefficients to the error we minimize. α sets how strong the penalty is.

$$\text{penalized error} = \text{MSE} + \alpha \times \text{penalty}$$

Ridge

$$\text{MSE} + \alpha \sum_j \beta_j^2$$

penalty: the sum of the squared coefficients

Lasso

$$\text{MSE} + \alpha \sum_j |\beta_j|$$

penalty: the sum of the absolute coefficients

α = 0 gives back ordinary least squares (OLS).

Lasso vs Ridge: Summary

Ridge

- **Shrinks** the estimate values.
- Useful when there are **correlated input columns**.

Lasso

- Does **feature selection**.
- Useful when there are **unrelated input columns**.

- The quality of the solution depends on the value of the hyper-parameter α.
- Selecting a good value of α can give better results than OLS.

How do we identify the best value of the hyper-parameter?

Train–Testing



Proportions are illustrative.

- Two datasets are available: **training** and **testing**.
- Build the model on the training data; test it on the testing data, usually the smaller set.
- The aim is to test the model's **generalizability**.
- Variation: training–testing–validation datasets.
  - the goal is to reduce over-fitting; training is done more rigorously
  - the validation set checks the model's accuracy before the actual test

Cross-Validation for Parameter Selection

Used when there is no predefined training / validation / testing data: partition the data repeatedly and pick the parameter that validates best.

Holdout

one split into training and validation



k-fold

k equal folds; each is the validation set once



Case Study 1: Real Estate Prices

Case Study 1

Historical real estate prices per unit area for a city, with six input features. The data is in `Regression-4.csv`.

Column	Description	Unit
X1	transaction date	year (e.g. 2013.250)
X2	house age	year
X3	distance to the nearest MRT station	meter
X4	number of nearby convenience stores	integer
X5	geographic coordinate, latitude	degree
X6	geographic coordinate, longitude	degree
Y	house price per unit area (output)	

Hypothesis

The input variables are linearly related to the output variable.

Objective

Identify each input's relationship with Y: run a regression analysis and estimate the best coefficients. Not every input need be related.

Step 1: Read and Describe the Data

```python
import pandas as pd
df = pd.read_csv('data/Regression-4.csv',
                 delimiter = ',')
display(df)
display(df.info())
df.describe().T
```

414	7	0
rows	columns	missing values

From df.info(): 414 non-null in every column.

	mean	std	min	50%	max
X1 date	2013.15	0.28	2012.67	2013.17	2013.58
X2 age	17.71	11.39	0.00	16.10	43.80
X3 MRT dist.	1083.89	1262.11	23.38	492.23	6488.02
X4 stores	4.09	2.95	0	4	10
X5 latitude	24.969	0.012	24.932	24.971	25.015
X6 longitude	121.533	0.015	121.474	121.539	121.566
Y price	37.98	13.61	7.60	38.45	117.50

Worth noticing

- Scales differ hugely: X3 runs into the thousands (meters), while X5 and X6 vary in the third decimal.
- X3's mean (1,084) is twice its median (492): right-skewed.
- Y's maximum, 117.5, is far above its 75th percentile (46.6): a possible outlier.

Steps 2–3: Split, Then Scale

```python
from sklearn.model_selection import train_test_split
X = df.iloc[:,:-1].values     # every column except the last one
y = df.iloc[:, -1].values     # the last column
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```




```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(np.c_[X_train,y_train])

A_train = scaler.transform(np.c_[X_train,y_train])
X_train = A_train[:,:-1]
y_train = A_train[:,-1]

A_test = scaler.transform(np.c_[X_test,y_test])
X_test = A_test[:,:-1]
y_test = A_test[:,-1]
```

Step 4: Fit OLS, Ridge and Lasso

```python
from sklearn.metrics import mean_squared_error

## OLS
from sklearn.linear_model import LinearRegression
reg1 = LinearRegression(fit_intercept=False).fit(X_train, y_train)
y_pred1 = reg1.predict(X_test)
print('The MSE using OLS is:', mean_squared_error(y_test, y_pred1))
print('The best values for the estimates are :',
      np.round(reg1.intercept_,2), np.round(reg1.coef_,2).tolist())

## Ridge
from sklearn.linear_model import RidgeCV
reg2 = RidgeCV(alphas=[1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3],
               fit_intercept=False, cv=10).fit(X_train, y_train)
y_pred2 = reg2.predict(X_test)
print('The MSE using Ridge is:', mean_squared_error(y_test, y_pred2))

## Lasso
from sklearn.linear_model import LassoCV
reg3 = LassoCV(alphas=[1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3],
               fit_intercept=False, cv=10, random_state=0).fit(X_train, y_train)
y_pred3 = reg3.predict(X_test)
print('The MSE using Lasso is:', mean_squared_error(y_test, y_pred3))
```

RidgeCV and **LassoCV** try seven α values from 0.001 to 1000 with **10-fold** cross-validation on the training data, then refit with the best.

Results: Test-Set MSE

OLS	Ridge	Lasso
0.3841	0.3794	0.3789
		lowest

Output

```
The MSE using OLS is: 0.3840946602349499
The best values for the estimates are : 0.0 [0.12, -0.2, -0.46, 0.23, 0.22, -0.06]
The MSE using Ridge is: 0.3793714233792296
The MSE using Lasso is: 0.37887845991254415
```

Reading it

- Both penalized models beat OLS on the test set; Lasso is lowest.
- The gains are small (about 1.4%): the six inputs do not over-fit badly.
- OLS prints an intercept of 0.0 because fit_intercept=False on centred data.

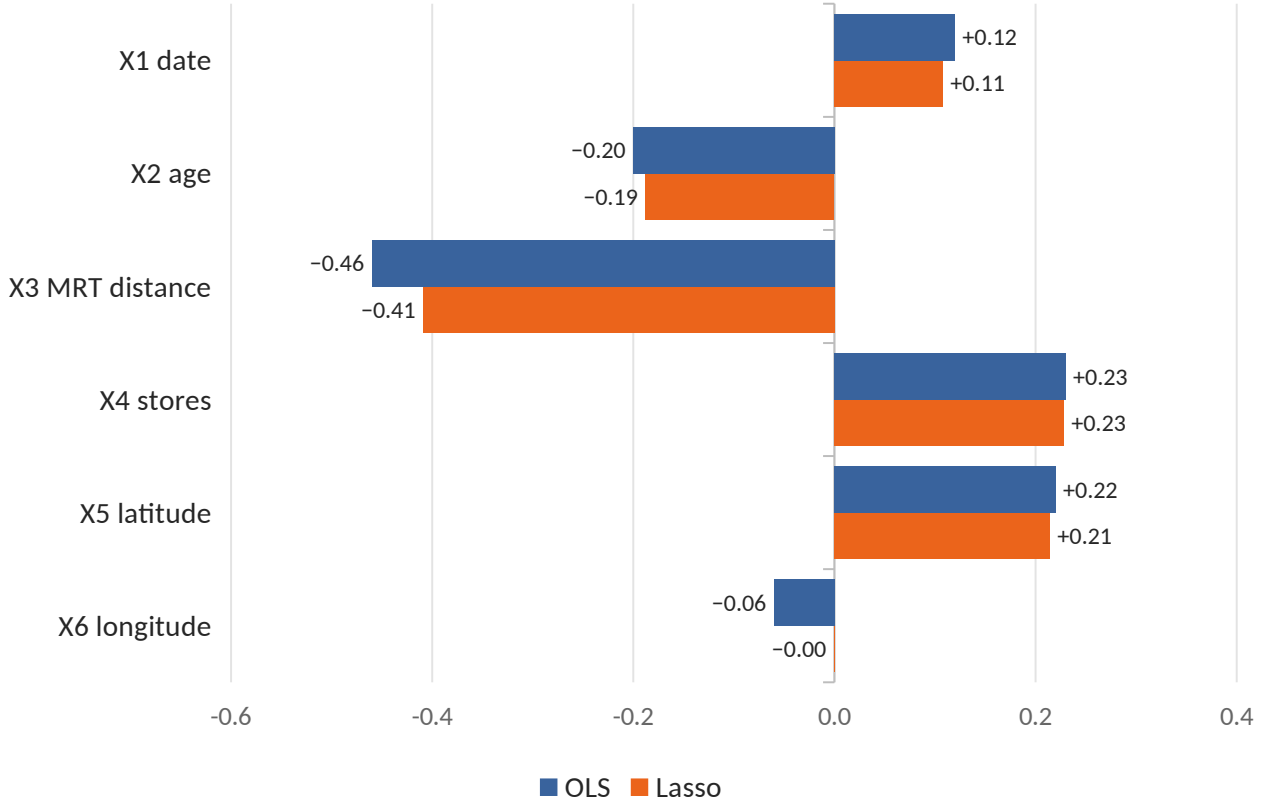
The Best Estimates: OLS vs Lasso

```python
print('The best penalty coefficient is:', reg3.alpha_)
print('The best coefficient estimates are:', reg3.coef_)
```

0.01

best Lasso α

	Lasso coef.
X1	+0.1080
X2	−0.1878
X3	−0.4089
X4	+0.2283
X5	+0.2144
X6	−0.0004



Summary of Case Study 1

1. Lasso performs better than OLS and Ridge for this data.

2. Input X6 (longitude) seems to have no relationship with Y.

3. Given new values of X1 to X5, the Lasso model reg3 can predict the price Y.

End of lecture